

Lessons from Writing Plugin Skills

What the adversarial review taught us

conmage

2026-05-10

Context

The `create-study` skill for the `claude-conscript-plugin` went through two adversarial reviews: one checking for overengineering, one simulating a student user. The initial version was 376 lines across 3 files; the revised version is 123 lines. This document captures what went wrong and the resulting principles.

Scope: These lessons are drawn from `congame/conscript` plugin skills. They likely generalize but have not been tested beyond this context.

Lessons

1. Don't restate loaded skills

The initial `create-study` repeated content from `coding`, `conscript-coding`, and `study-review` — skills it explicitly loads at the top. This created 2–3x the necessary length and a drift risk: if the originals update, the copies become contradictory.

Rule: If your skill says “load skill X,” trust that X is loaded. Don't paraphrase it.

2. Match existing style

The plugin's original skills (by Bogdan Popa) use flat numbered sections, terse language, and no meta-commentary. The initial `create-study` introduced “Phase A / Phase B” labels, hedged language (“adapt based on answers”), and parenthetical examples. The adversarial reviewer flagged this as 2–3x the word count for equivalent instruction complexity.

Rule: Read other skills in the plugin first. Match their density.

3. Don't over-script user interactions

The initial version prescribed a 4-step, 15-question interview with parenthetical examples for each question. The existing `study-review` achieves similar interaction in one line: “ask the user what the experimental task is.”

Rule: State the intent, not the script. Claude adapts better to the user when given goals than when given a rigid Q&A.

4. Don't force two invocations

The initial design required running `/create-study` twice — once for setup, once for actual creation. The usability reviewer called this the biggest friction point.

Rule: If there's a one-time setup step, do it and continue. Don't stop and make the user re-invoke.

5. Define jargon

The usage guide mentioned “conscript,” “congame,” “-with-admin variant,” and “bot models” without defining any of them. The usability reviewer (simulating a student) flagged all four as blocking.

Rule: If the audience includes newcomers, define domain terms. Experts won't mind; newcomers will be stuck.

6. Verify framework claims before writing

The bot testing analysis stated “bots bypass rendering” and “bots can't read pages.” This was wrong — congame bots use Marionette to drive headless Firefox with full DOM access. The error propagated into recommendations (“don't try to make bots do content verification”) that would have misdirected development effort.

Rule: Before writing “the framework can't do X,” grep the source code. Check `provide` forms, imported modules, and test files for evidence.

7. Separate reference from report

The `bot-testing-analysis-detail.md` was written as a reference doc but sent to Marc as if it were a report. Reference docs (`.md`) should be dense and tool-readable. Reports (`.qmd` → PDF) should have prose context for human readers.

Rule: Know your audience for each file. Write accordingly.